Лабораторная работа №5. Синхронизация сигналов: DQPSK, M-PSK

GNU Radio 3.10 (radioconda). Все схемы открываются в GNU Radio Companion (`gnuradio-companion файл.grc`). Каждая схема проверена компилятором `grcc`, а все симуляционные схемы ещё и запущены. Численные результаты ниже получены скриптом tools/sim_lab5.py: он собирает те же блоки, только без графики. Файлы генерируются скриптом tools/make_lab5.py, поэтому любую схему можно пересобрать.

Пункт	Файл
1.1 – 1.9.2	lab5_1_dqpsk.grc
1.9.1 (компенсация)	lab5_1_9_1_dqpsk_heterodyne.grc (ручной гетеродин), lab5_1_9_1_dqpsk_fll.grc (FLL Band-Edge, сравнение), lab5_1_9_1_dqpsk_fll_simple.grc (то же без Random Source и Symbol Sync)
1.9.3	lab5_1_9_3_dqpsk_phase.grc
1.9.4	lab5_1_9_4_dqpsk_one_symbol.grc
1.9.5	lab5_1_9_5_dqpsk_sdr.grc
2.1 – 2.7.1	lab5_2_1_bpsk.grc
2.7.2	lab5_2_7_2_bpsk_costas.grc
2.7.3	lab5_2_7_3_bpsk_ted.grc
2.7.4	lab5_2_7_4_bpsk_osps.grc
2.7.5	lab5_2_7_5_bpsk_resampler.grc
2.7.6	lab5_2_7_6_qpsk.grc
2.7.7	lab5_2_7_7a_qpsk_foffset_costas.grc (только Костас), lab5_2_7_7b_qpsk_fll.grc (FLL Band-Edge)
2.7.8	lab5_2_7_8_qpsk_sdr.grc
2.7.9	lab5_2_7_9_qpsk_two_ones.grc
2.7.10	lab5_2_7_10_8psk_two_sixes.grc

SDR-схемы собраны под стенд из ЛР4: передатчик HackRF (`osmosdr_sink`, `hackrf=0`) и приёмник RTL-SDR (`osmosdr_source`, `rtl=0`), несущая 433,92 МГц. Если на стенде другое устройство, поменяйте строку `args`.

> В симуляционные схемы добавлен блок **Throttle**: без SDR ничто не ограничивает скорость потока, и без этого блока процессор загружается на 100 %. В SDR-схемах Throttle нет, там темп задаёт само устройство.

Числовые параметры вместо блоков Import и Variable

В схемах нет блоков Import (`pi`, `cmath`, `firdes`): все величины записаны числами. Модуль `firdes` GRC импортирует в сгенерированный скрипт сам.

Значение	Откуда	Смысл
`0.0628`	2π/100	полоса петли Костаса и PLL, рад/символ (рад/отсчёт для PLL): петля отслеживает изменения фазы примерно за 100 символов
`0.00628`	2π/1000	полоса петли FLL Band-Edge, рад/отсчёт: в 10 раз уже, чтобы оценка частоты не «дрожала»
`0.251`	2π/25	расширенная полоса петли Костаса для 8PSK (п. 2.7.10)
`57.2958`	180/π	перевод радиан в градусы (п. 1.9.3)
`0.707`	cos 45° = 1/√2	координата точки QPSK на диагонали при амплитуде 1
`0.924`, `0.383`	cos 22,5°, sin 22,5°	координаты точек 8PSK

Коэффициенты RRC-фильтра (схемы п. 1.9.1 с FLL) записаны прямо в поле Taps фильтров, без переменной `rrc_taps`:

- передатчик (Interpolating FIR Filter, Interpolation = sps): `firdes.root_raised_cosine(sps, sps, 1.0, alpha, 111)`;
- приёмник (Decimating FIR Filter, Decimation = 1): `firdes.root_raised_cosine(1, sps, 1.0, alpha, 111)`.

Аргументы функции по порядку:

1. **gain** — коэффициент усиления. На передаче он равен `sps`, потому что интерполятор вставляет между символами нули и без этого амплитуда упала бы в sps раз. На приёме он равен 1.

2. **sampling_freq** = `sps` — частота дискретизации в единицах символьной скорости.
3. **symbol_rate** = 1 — символьная скорость в тех же единицах, т.е. на символ приходится sps отсчётов.
4. **alpha** — коэффициент скругления (0,35).
5. **ntaps** = 111 = 11·sps + 1 — длина фильтра: импульс охватывает 11 символов. Длина нечётная, поэтому задержка фильтра (111 − 1)/2 = 55 отсчётов — целое число.

Числами эти 111 коэффициентов не записывают: это длинный список, и он изменился бы при любой правке `sps` или `alpha`.

Часть 1. DQPSK

1.1 – 1.8. Сборка схемы (рис. 1)

№	Блок	Параметры и назначение
1	Vector Source (byte)	`(0, 1, 2, 3)`, Repeat = Yes. По п. 1.2 передаётся **весь алфавит** DQPSK
2, 3	UChar To Float → QT GUI Time Sink «Переданное»	100 точек: так видны передаваемые символы
4	Differential Encoder	Modulus = **4**, т.е. M для QPSK: `y[n] = (x[n] + y[n-1]) mod 4`
5	Chunks to Symbols	таблица `[0.707+0.707j, -0.707+0.707j, -0.707-0.707j, 0.707-0.707j]`, Dimension = 1. Это точки на 45°, 135°, 225°, 315°; индекс k даёт поворот на k·90°
6	Channel Model	Noise Voltage = `n*1e-3`, Frequency Offset = `f/samp_rate`
7	QT GUI Range `n`	СКО шума в мВ (0 … 1000). В исходной схеме ползунок кончался на 100, но ошибки появляются только примерно от 200 мВ, поэтому диапазон расширен
8	QT GUI Range `f`	расстройка в Гц (−10 … +10 кГц). **В Channel Model частота задаётся относительной**, поэтому `f/samp_rate`
9, 10	Virtual Sink / Virtual Source	одинаковый Stream ID = `dqpsk`
11	Differential Phasor	`y[n] = x[n]·x*[n−1]`: выход несёт **приращение фазы** между соседними символами
12	QT GUI Constellation Sink (2 входа)	вход 0: до Differential Phasor, вход 1: после него
13	Constellation Decoder	Constellation Object = `dqpsk`

№	Блок	Параметры и назначение
14	Constellation Object `dqpsk`	Variable Constellation; Symbol Map `[0,1,2,3]`; точки `[1, 1j, -1, -1j]`; Rotational Symmetry 4; Dimensionality 1; Normalization Amplitude
15, 16	UChar To Float → Time Sink «Принятое»	

Почему созвездие на приёме другое. Differential Phasor возвращает не саму точку, а разность фаз. Приращение k·90° превращается в точку `e^{jk·90°}`, то есть в 1, j, −1 или −j. Поэтому в объекте созвездия стоят именно эти точки, а не точки передатчика (45°, 135°, …).

1.9. Запуск и анализ

Принятая последовательность совпадает с переданной (0, 1, 2, 3, …); в симуляции SER = 0. На входе 0 созвездия видны 4 точки QPSK на диагоналях, на входе 1 — 4 точки на осях (0°, 90°, 180°, −90°).

- **Достоинства DQPSK:** не нужно восстанавливать абсолютную фазу несущей. Постоянный фазовый сдвиг канала сокращается в `x[n]·x*[n−1]`, а неоднозначность фазы на 90° исчезает. Схема устойчива к небольшой расстройке частоты.
- **Недостатки:** шум действует дважды (на текущий и на предыдущий символ), поэтому помехоустойчивость примерно на 2–3 дБ хуже когерентной QPSK. Одна ошибка порождает две ошибочные разности.

1.9.1. Расстройка частоты (ползунок `f`)

Созвездие до Differential Phasor начинает вращаться и при большой расстройке размывается в кольцо. Созвездие после Phasor только **поворачивается на постоянный угол** `Δϕ = 360°·f/Rs`. В схеме 1 отсчёт на символ, поэтому Rs = 2 МГц.

Симуляция: 1 кГц → 0,2°, 5 кГц → 0,9°, 10 кГц → 1,8°, SER = 0 во всех случаях. Ошибки начнутся, когда поворот превысит 45°, то есть при f > Rs/8 = 250 кГц. Это главное преимущество DQPSK: расстройку, которая полностью разрушила бы когерентный приём, она переносит без ошибок.

Откуда берётся фазовый сдвиг. Расстройка f умножает сигнал на `e^{j2πf·n/fs}`, и фаза растёт на `2πf/fs` за каждый отсчёт. Differential Phasor вычисляет `y[n] = x[n]·x*[n−1]`. Постоянная фаза сокращается, а набег за один символ остаётся: `y[n] = (полезное приращение)·e^{j2πf/Rs}`. Поэтому все точки повёрнуты на одинаковый угол `360°·f/Rs`.

Компенсация гетеродином (lab5_1_9_1_dqpsk_heterodyne.grc): после Virtual Source добавлены **Signal Source** (complex, частота `-f`) и **Multiply**. Это цифровой гетеродин: умножение на `e^{−j2πf·n/fs}` возвращает спектр на нулевую частоту. Симуляция: при 2, 10 и 50 кГц поворот был 0,36°, 1,8° и 9° (точно по формуле), а с гетеродином стал 0,000°. Начальная фаза гетеродина не важна: постоянную фазу убирает сам Differential Phasor. Ограничение: здесь гетеродин знает точное значение `f`. В реальном приёмнике расстройка неизвестна и дрейфует, а ошибка гетеродина Δf снова даёт поворот `360°·Δf/Rs`. Поэтому гетеродин нужно подстраивать автоматически по оценке частоты (АПЧ). Именно так устроены FLL Band-Edge и петля Костаса: внутри у них управляемый генератор (NCO), то есть гетеродин с обратной связью. **Реальный гетеродин с АПЧ — FLL Band-Edge** (lab5_1_9_1_dqpsk_fll.grc). FLL оценивает частоту по краям спектра RRC-сигнала, поэтому схема переделана:

- передатчик: случайные символы 0…3 → Differential Encoder → Chunks to Symbols → **Interpolating FIR Filter** с RRC-импульсом (`sps = 10`, `alpha = 0.35`), Rs = 200 кБод;
- канал: ползунок расстройки ±50 кГц и ползунок шума;

- приёмник в две ветви от одного Virtual Source: **(а) без АПЧ**: Symbol Sync → Differential Phasor → Decoder; **(б) с АПЧ**: **FLL Band-Edge** (Loop BW `0.00628`) → Symbol Sync → Differential Phasor → Decoder;
- на общих графиках: созвездие после Phasor (1 — без FLL, 2 — с FLL), принятые символы обеих ветвей, спектр до и после FLL (видно, как FLL возвращает сдвинутый спектр в центр).

Символов в секунду теперь в 10 раз меньше, поэтому поворот `360°·f/Rs` в 10 раз больше, чем в основной схеме. Результаты симуляции (случайные данные, без шума):

Расстройка	без FLL: поворот / доля ошибок	с FLL: поворот / доля ошибок
10 кГц	18° / 0	0,02° / 0
20 кГц	36° / 0	0,02° / 0
30 кГц	54° (> 45°) / **≈ 75 %**	0,02° / 0
40 кГц	72° / **≈ 75 %**	0,02° / 0

Без АПЧ созвездие поворачивается пропорционально расстройке. После 45° точки переходят в чужие области решений, и почти все символы принимаются неверно. С FLL поворот остаётся около нуля при любой расстройке, потому что FLL сам находит частоту и подстраивает свой генератор (NCO). Знать `f` заранее ему не нужно, в отличие от ручного гетеродина.

Упрощённый вариант (lab5_1_9_1_dqpsk_fll_simple.grc):

- источник, как в исходной схеме, — **Vector Source** `(0, 1, 2, 3)`, Random Source не нужен;
- вместо Symbol Sync стоят **RRC-фильтр на приёме** (согласованный фильтр) → **Delay** → **Keep One in N** (N = sps): берётся каждый 10-й отсчёт;

- в блоках Delay заданы значения, при которых отсчёт берётся в центре символа.

Результаты симуляции, поворот созвездия после Phasor:

Расстройка	без FLL	с FLL
0	0°	0,01°
10 кГц	18°	0,01°
20 кГц	36°	0,01°
30 кГц	54° → ошибки (точки в чужих секторах)	0,01°
40 кГц	72° → ошибки	0,01°

Без Symbol Sync схема работает только в симуляции: такты передатчика и приёмника там совпадают, и момент отсчёта не уплывает. Для SDR нужна полная схема с Symbol Sync.

Другие способы: оценить угол по выходу Phasor как `arg(mean(y⁴))/4` и повернуть созвездие на обратный угол, либо подстроить частоту SDR (ppm, центральная частота).

1.9.2. Шум (ползунок `n`, f = 0)

Результаты симуляции (вероятность ошибки на символ):

Noise Voltage	100 мВ	200 мВ	300 мВ	400 мВ	500 мВ	700 мВ
SER	0	$1{,}5\cdot10^{-4}$	$1{,}1\cdot10^{-2}$	$6{,}2\cdot10^{-2}$	0,14	0,31

Ошибки декодирования начинают появляться при СКО шума **около 200 мВ**; при 300 мВ их уже видно на осциллограмме. Облака точек на созвездии расширяются и начинают заходить за границы решений (±45°).

1.9.3. Вывод фазового приращения (lab5_1_9_3_dqpsk_phase.grc)

Noise Voltage = 0. После Differential Phasor добавлены блоки: **Complex to Arg** (фаза в радианах) → **Multiply Const** `57.2958` (= 180/π, перевод в градусы) → **Time Sink «Фазовое**

приращение, град». На осциллограмме повторяется последовательность **0°, 90°, 180°, −90°** (−90° = 270°): символ k даёт приращение k·90°.

1.9.4. Передача одного символа (lab5_1_9_4_dqpsk_one_symbol.grc)

В Vector Source задан вектор `(2,)`. Дифференциальный кодер накапливает 0, 2, 0, 2, …, поэтому передатчик **всё время чередует две противоположные точки** (скачок фазы 180°). Созвездие на входе 0 выглядит как две точки, хотя символ один. Приращение фазы постоянно и равно 180° (точка −1 после Phasor), и декодер всё время выдаёт «2». Вывод: в DQPSK информацию несёт не положение точки, а **переход между точками**.

1.9.5. Схема для оборудования (lab5_1_9_5_dqpsk_sdr.grc)

Изменения:

- Channel Model, Virtual Sink/Source и Throttle удалены.
- Передатчик: Chunks to Symbols → **Repeat (sps = 10)** → Multiply Const 0.5 → **osmosdr Sink (HackRF)**. Повтор нужен потому, что при 1 отсчёте на символ приёмник с собственным тактовым генератором не может выбрать момент отсчёта. Rs = 2 МГц / 10 = 200 кБод.
- Приёмник: **osmosdr Source (RTL-SDR)** → AGC → **Symbol Sync** (Gardner, sps = 10, 1 отсчёт на выходе) → Differential Phasor → …
- Ползунки TX gain, RX gain, RX ppm и спектр принятого сигнала.

Отдельный частотный синхронизатор не нужен. Уход частоты RTL-SDR (~20 ppm ≈ 9 кГц на 433 МГц) даёт поворот

приращения всего на 360°·9 кГц / 200 кГц ≈ 16°, а это меньше 45°.

Часть 2. M-PSK

Переменные: `samp_rate = 2M`, `sps = 10`, `alpha = 0.4`, `Rs = samp_rate/sps = 200k`.

2.1 – 2.6. Схема (lab5_2_1_bpsk.grc)

Передатчик (рис. 2–3): Random Uniform Source (byte, 0…2, т.е. биты 0/1) → Unpacked to Packed (1 бит/чанк, MSB) → **Constellation Modulator** (Constellation `psk` = BPSK, Differential Encoding = No, Samples/Symbol = `sps`, Excess BW = `alpha`) → Throttle → Channel Model → Frequency Sink (FFT 8192), Time Sink (1024 точки) и Virtual Sink `psk`.

Приёмник (рис. 4–5): Virtual Source `psk` → Constellation Sink (до синхронизации) и **Symbol Sync** (Early-Late, sps, TED gain 1, Loop BW `alpha`, Damping 1, Max Dev 1.5, Output SPS 1, MMSE 8 tap) → Eye Sink (2048 точек, sps 10) и **Multiply (вход 0 и вход 1 от Symbol Sync, т.е. возведение в квадрат) → PLL Carrier Tracking** (Loop BW `0.0628` = 2π/100, Max/Min ±0,7 рад/отсчёт) → Constellation Sink.

Смысл грубой синхронизации: у BPSK фаза равна 0 или π, после возведения в квадрат она становится 0 или 2π. Модуляция снимается, остаётся чистая несущая на удвоенной частоте, и PLL отслеживает её фазу.

2.7. Полоса сигнала

Расчёт для RRC-спектра: **B = Rs·(1 + α) = 200 кГц · 1,4 = 280 кГц**, то есть ±140 кГц относительно центра. По Frequency Sink полоса по уровню, где спектр резко спадает, около 280 кГц, что совпадает с расчётом.

2.7.1. Глазковая диаграмма и созвездие

- После Symbol Sync с Output SPS = 1 на каждый символ приходится один отсчёт. Eye Sink считает, что отсчётов на символ 10, и строит «глаз» из 20 разных символов. Реального глаза нет: видны только два уровня ±1 (горизонтальные линии). Это правильно для синхронизированного потока: все отсчёты взяты в центре символов. Ось времени при этом неверна: в Eye Sink указано 2 МГц, а реальная частота 200 кГц.
- Выходное созвездие после x^2 и PLL — **одна точка**: модуляция снята возведением в квадрат. Синхронизация несущей работает, но данные из такого сигнала уже не извлечь (+1 и −1 дают одно и то же). Отсюда название «грубая».

2.7.2. Точная синхронизация (lab5_2_7_2_bpsk_costas.grc)

Multiply и PLL Carrier Tracking заменены на **Costas Loop** (Loop BW `0.0628`, Order **2**). На выходе снова **две точки ±1**, модуляция сохраняется. Вывод: грубая схема (x^2 + PLL) проста, но разрушает информацию и имеет неоднозначность фазы на π. Петля Костаса синхронизирует фазу прямо по модулированному сигналу. Неоднозначность π у неё остаётся, она устраняется дифференциальным кодированием.

2.7.3. Сравнение TED (lab5_2_7_3_bpsk_ted.grc)

Шум 50 мВ (ползунок `noise` позволяет менять его до 1 В). Фазовое СКО точек на выходе петли Костаса (меньше — лучше):

TED	шум 50 мВ	шум 100 мВ
Mueller and Müller	1,94°	3,83°
Modified M&M	1,92°	**3,81°**
Zero Crossing	1,95°	3,87°
Gardner	2,12°	3,94°
Early-Late	2,38°	**5,38° (хуже всех)**
Signal × Slope ML	**1,89°**	4,17°
Signum × Slope ML	1,94°	4,21°

Early-Late (исходный вариант) проигрывает заметно. Остальные отличаются в пределах ~0,3°. **Выбран Gardner.** Он отстаёт от лучших всего на ~0,1°, зато не требует решений о символах (non-data-aided) и не зависит от фазы несущей. Это важно, потому что Symbol Sync стоит **до** петли Костаса, а в п. 2.7.7 появится расстройка частоты. Решающие TED (M&M, Zero Crossing) при вращающемся созвездии работают плохо.

2.7.4. Output Samples/Symbol = sps (lab5_2_7_4_bpsk_osps.grc)

Теперь на выходе 10 отсчётов на символ, и Eye Sink (sps = 10, 2 МГц) показывает **настоящий глаз**: период 2 символа, раскрыв глаза в точке отсчёта, длительность символа **T = 1/Rs = 5 мкс = 10 отсчётов**. Созвездие при этом **размазано**: кроме оптимальных отсчётов в него попадают все промежуточные точки переходов, и петля Костаса работает хуже. **Оптимальное значение Output SPS = 1**: блок решений должен получать ровно один отсчёт в момент наибольшего раскрыва глаза. Значение sps полезно только для наблюдения глазковой диаграммы.

2.7.5. Тип интерполятора (lab5_2_7_5_bpsk_resampler.grc)

Output SPS = 1, TED Gardner, шум 50 мВ:

Interpolating Resampler	фазовое СКО	разброс амплитуды
MMSE, 8 tap FIR	2,12°	0,150
Polyphase Filterbank, no MF	2,12°	0,150
Polyphase Filterbank, MF	**0,69°**	**0,060**

Оставлен **PFB с согласованным фильтром (MF)**. В его параметре *PFB MF Taps* задан RRC-фильтр `firdes.root_raised_cosine(128, 128*sps, 1.0, alpha, 11*sps*128)`. Интерполятор одновременно выполняет согласованную RRC-фильтрацию: передатчик RRC + приёмник RRC = фильтр Найквиста (RC) без межсимвольной интерференции. Кроме того, согласованный фильтр подавляет внеполосный шум. Два других варианта согласованного фильтра не содержат.

2.7.6. QPSK (lab5_2_7_6_qpsk.grc)

Изменения: Constellation Object → **QPSK**; Costas Loop → **Order 4**; в Symbol Sync Loop Bandwidth `alpha` → **0.045**. Последнее изменение потребовалось после проверки: с полосой 0,4 (это двухсторонняя полоса относительно Rs, т.е. очень широкая) у QPSK заметен временной джиттер, фазовое СКО ≈ 10°. С 0,045 СКО ≈ 0,5°. На выходе четыре компактные точки на диагоналях, как и ожидалось. Полоса сигнала не изменилась (280 кГц), а скорость передачи удвоилась: 2 бита на символ.

2.7.7. Расстройка N·10 кГц

В Channel Model: Frequency Offset = `N*10e3/samp_rate`, переменная `N` = номер по списку (в файле N = 1, поставьте

свой).

- lab5_2_7_7a_qpsk_foffset_costas.grc: работает только петля Костаса.
- lab5_2_7_7b_qpsk_fll.grc: **перед** Symbol Sync добавлен **FLL Band-Edge** (Samples/Symbol `sps`, Rolloff `alpha`, Filter Size 44, Loop BW `0.00628`).

N (расстройка)	только Костас	FLL + Костас
1 (10 кГц)	0,7°	0,7°
3 (30 кГц)	4,1°	0,7°
5 (50 кГц)	10,4°	0,7°

У петли Костаса ограничена полоса захвата. С ростом расстройки растёт её динамическая ошибка, затем начинаются срывы. FLL Band-Edge оценивает частоту по энергии на краях спектра RRC (до символьной и фазовой синхронизации) и убирает расстройку, после чего Костас работает в идеальных условиях. Качество созвездия от N больше не зависит. **Улучшение.** Важная настройка: с полосой петли FLL `0.0628` (2π/100, значение из примеров) частота «дрожит», и фазовое СКО на выходе ≈ 24° (созвездие разваливается). Поэтому взята полоса `0.00628` (2π/1000).

2.7.8. QPSK через SDR (lab5_2_7_8_qpsk_sdr.grc)

Channel Model и Throttle заменены на Multiply Const 0.5 → **osmosdr Sink (HackRF)**. Приёмник: **osmosdr Source (RTL-SDR)** → **AGC** (нормирует уровень; FLL и Symbol Sync рассчитаны на амплитуду ≈1) → FLL Band-Edge → Symbol Sync → Costas (4). Порядок настройки: начать с TX gain = 0 и увеличивать, пока на спектре не появится сигнал на 20–30 дБ выше шума. Если созвездие вращается слишком быстро, скорректировать ppm. Добиться 4 устойчивых точек.

2.7.9. Передача двух «1» (lab5_2_7_9_qpsk_two_ones.grc)

Изменения:

- Random Source заменён на **Vector Source** `(1, 1)`, Unpacked to Packed → **2 бита/чанк** (один символ QPSK = 2 бита).
- Созвездие заменено на Variable Constellation с точками **по кругу по порядку** `[0.707+0.707j, -0.707+0.707j, -0.707-0.707j, 0.707-0.707j]` (углы 45°, 135°, 225°, 315°), Symbol Map `[0,1,2,3]`. Точки стоят на диагоналях, потому что петля Костаса 4-го порядка фиксирует созвездие именно там.
- **Differential Encoding = Yes** в модуляторе, на приёме **Constellation Decoder** → **Differential Decoder (4)** → Time Sink «Принятые символы»; рядом Time Sink «Переданные символы».

Зачем дифференциальное кодирование: петля Костаса может захватить созвездие с поворотом на 90°, 180° или 270°. Без него все символы сдвинутся по кругу: в симуляции без дифф. кодирования вместо «1» принималось «0». С дифф. кодированием принимается ровно «1, 1, 1, …» (N = 0 и N = 1).

Оценка качества и ограничение. Последовательность из одинаковых символов после дифф. кодирования означает поворот фазы на 90° на каждом символе, то есть спектр — **одна линия**, а не полоса. Поэтому:

- FLL Band-Edge не видит краёв спектра;
- частотный сдвиг на Rs/4 = 50 кГц неотличим от смены символа.

В симуляции при N ≥ 2 (20 кГц) часть символов принимается как «2». Для реальных систем поэтому перед модулятором

ставят **скремблер**, делающий данные случайными. Для проверки по п. 2.7.9 используйте N = 0…1.

2.7.10. 8PSK, два символа «6» (lab5_2_7_10_8psk_two_sixes.grc)

Изменения относительно 2.7.9: Vector Source `(6, 6)`; Unpacked to Packed → **3 бита/чанк**; созвездие из 8 точек на углах 22,5° + k·45° (`0.924+0.383j, 0.383+0.924j, …`); Costas Loop → **Order 8**; Differential Decoder → **модуль 8**. **Изменение параметров синхронизации:** при полосе петли Костаса `0.0628` часть символов принималась как «7» (петля 8-го порядка не успевает за остаточной частотой). Полоса расширена до `0.251` (2π/25), после чего принимаются ровно «6, 6, 6, …» (N = 0…1, в том числе с шумом 50 мВ). Ограничение то же, что в 2.7.9, только жёстче: неотличимый сдвиг частоты здесь Rs/8 = 25 кГц.

Сравнение помехоустойчивости QPSK и 8PSK. При той же энергии символа расстояние между соседними точками равно `2·sin(π/M)`: 1,41 для QPSK и 0,77 для 8PSK. Граница решения у QPSK ±45°, у 8PSK ±22,5°. Чтобы получить ту же вероятность ошибки, 8PSK нужно примерно на **3,6 дБ** больше Eb/N0 (≈ на 5,3 дБ больше Es/N0 при той же вероятности ошибки на символ). Платой за это 8PSK передаёт 3 бита на символ вместо 2 в той же полосе. Кроме того, 8PSK чувствительнее к фазовому джиттеру синхронизаторов, поэтому пришлось менять полосу петли Костаса.

Контрольные вопросы (кратко)

1. **FLL Band-Edge.** Два полосовых фильтра стоят на краях спектра RRC-сигнала (±Rs(1+α)/2). Разность их выходных мощностей пропорциональна сдвигу спектра, этот сигнал ошибки управляет NCO через петлевой фильтр.

Параметры: samples/symbol, rolloff (α), filter size, loop bandwidth.

2. **Синхронизаторы:** частотный (FLL), символьный (тактовый; Symbol Sync — интерполятор, TED, петлевой фильтр 2-го порядка, управляющий дробной задержкой) и фазовый (PLL, Costas). Symbol Sync интерполирует отсчёты в оценённый момент, TED считает ошибку, петля подстраивает период и фазу отсчётов.
3. **TED:** Mueller & Müller и Modified M&M (по решениям, 1 отсчёт/символ), Zero Crossing (по решениям, 2 отсчёта), Gardner (без решений, 2 отсчёта, не зависит от фазы несущей), Early-Late (по разности ранний/поздний), ML Signal/Signum × Slope (по производной согласованного фильтра), а также специальные TED для GMSK.
4. **Синхронизация несущей:** возведение в степень M + PLL (просто, но шум усиливается в M раз и модуляция снимается), петля Костаса (работает по модулированному сигналу, неоднозначность 2π/M), решающая обратная связь DD-PLL (точна при высоком ОСШ, при ошибках хуже), FLL Band-Edge (широкий захват по частоте, но не устраняет фазу), пилот-сигналы/преамбулы (надёжно, но тратят ресурс).
5. **Петля Костаса для BPSK.** Сигнал умножается на cos и sin от NCO, получаются ветви I и Q. Каждая ветвь проходит ФНЧ, затем их произведение $I \cdot Q \propto \sin(2\Delta\phi)$ даёт ошибку, не зависящую от знака символа. Петлевой фильтр подстраивает NCO, пока Q не станет 0, и данные остаются в ветви I.